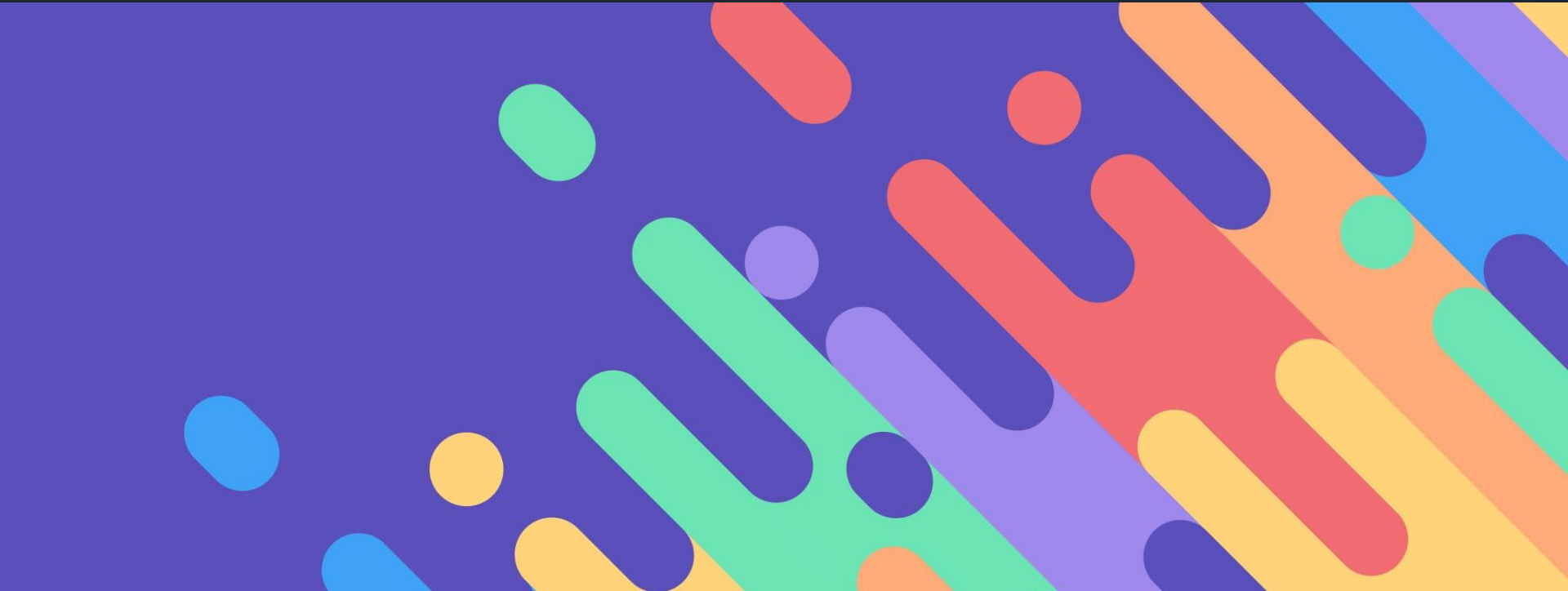


Language Models & Contextual Word Embeddings

An abstract graphic featuring a variety of colorful shapes including circles, ovals, and elongated rectangles in shades of purple, blue, green, red, orange, and yellow, scattered across a dark blue background.

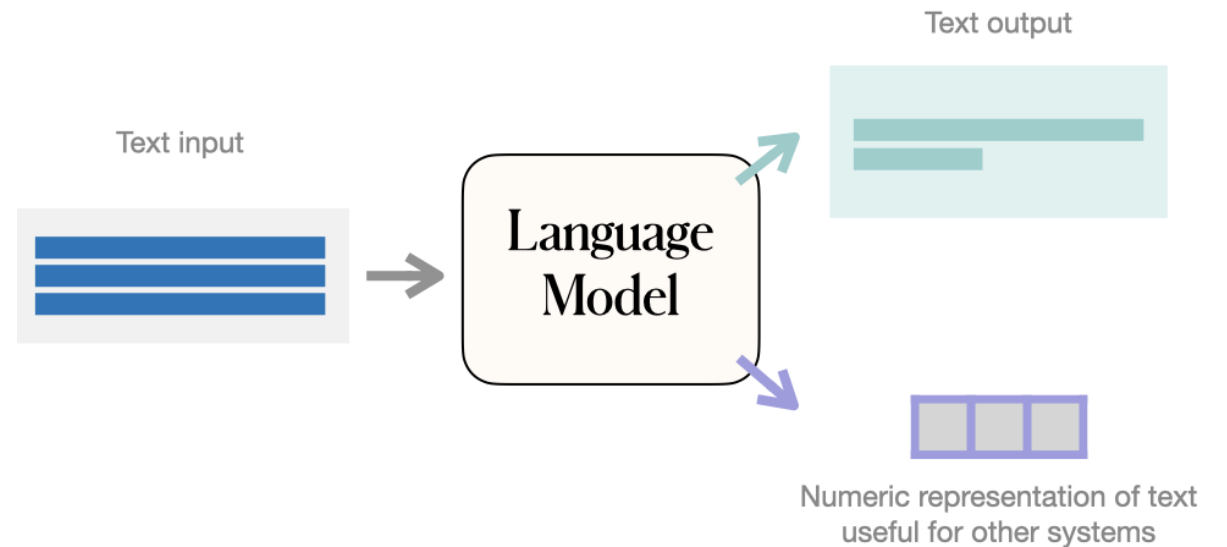
DIGITAL LINGUISTICS AND
LANGUAGE PROCESSING

Structure

- Fundamental concepts of language models
- State-of-the-art Transformer architecture
- Transformer variants
- Hands-on (Later in the afternoon): Visualization tool

What is a Language Model?

- A Language Model is an AI system trained to understand, generate, and predict human language.
- Language models are trained on **large text datasets** to learn grammar, syntax, and meaning.
- Trained with **self-supervised approach**: Model predicts the missing or next word
- Versions: Probabilistic, rule-based, knowledge-graph based, etc.



Source: <https://radical.vc/language-is-the-next-great-frontier-in-ai/>

Probabilistic Language Models

Goal: compute the probability of a sentence or sequence of words

- $P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$

Related task 1: probability of an upcoming word

- $P(w_5 | w_1, w_2, w_3, w_4)$

Related task 2: probability of missing words

- $P(? , w_2, ? , w_4, w_5)$

Probabilistic Language Models

The Goal: Assign a Probability to a Sequence

Machine Translation

- Example: "high winds tonight" vs. "large winds tonight"
- Probability Comparison: $P(\text{"high winds tonight"}) > P(\text{"large winds tonight"})$

Spell Correction

- Example: "The office is about fifteen minuets from my house"
- Probability Comparison: $P(\text{"about fifteen minutes from"}) > P(\text{"about fifteen minuets from"})$

Speech Recognition

- Example: "I saw a van" vs. "eyes awe of an"
- Probability Comparison: $P(\text{"I saw a van"}) \gg P(\text{"eyes awe of an"})$

Early Statistical LMs

- Early LMs were based on **estimating n-gram probabilities**.

<s> I am Sam </s>

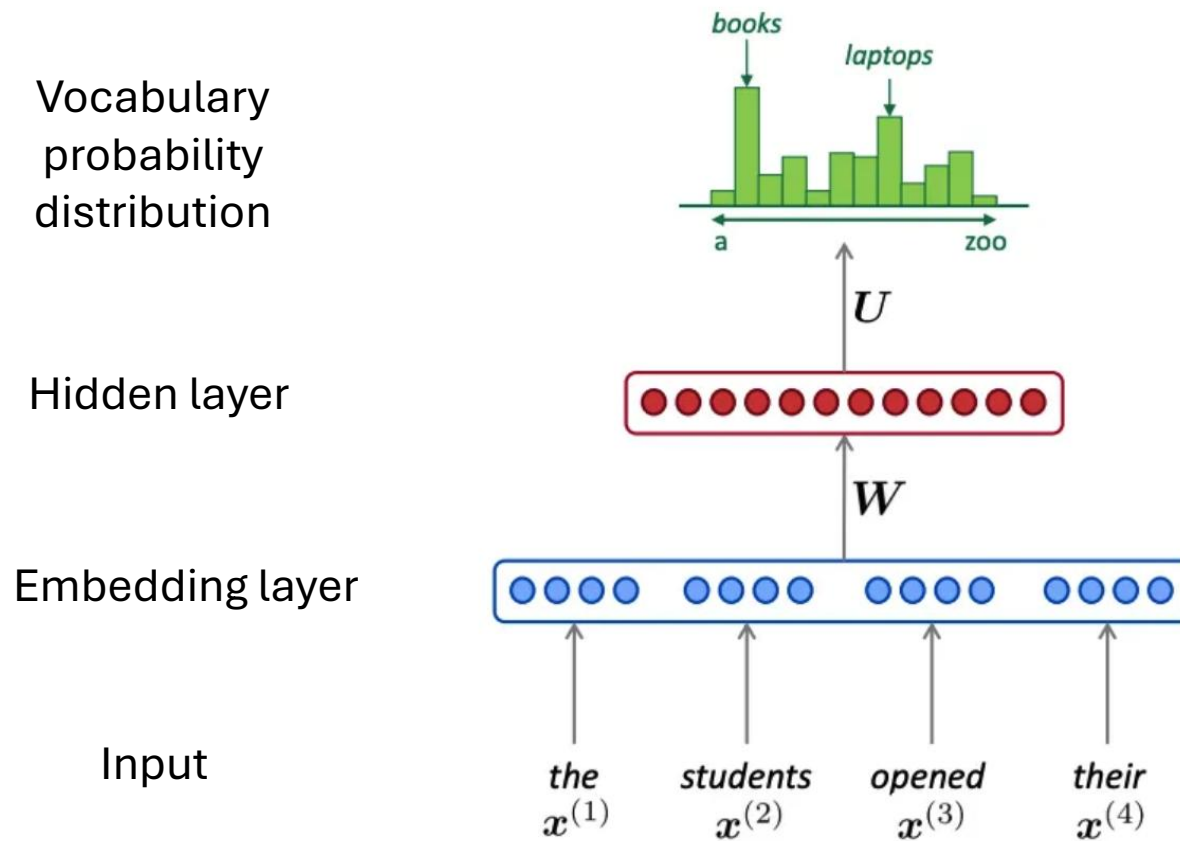
<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

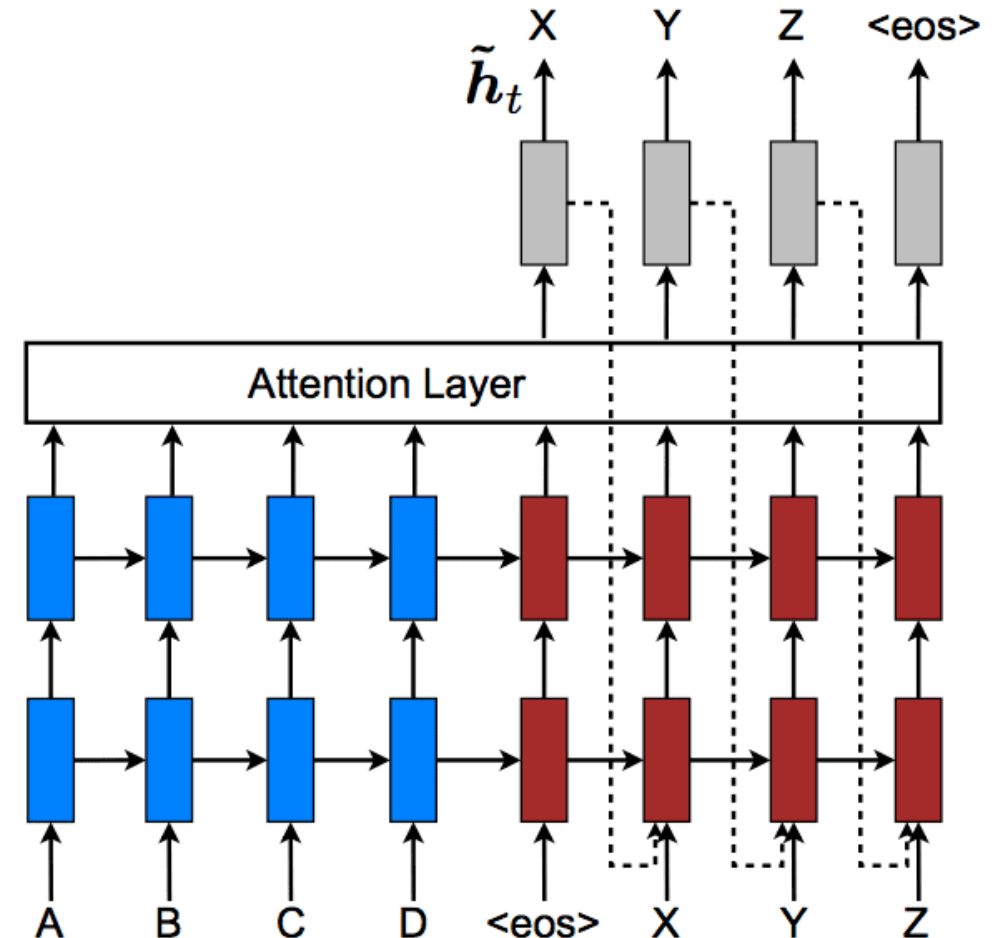
$$\begin{array}{lll} P(\text{I} | \text{<s>}) = \frac{2}{3} = .67 & P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33 & P(\text{am} | \text{I}) = \frac{2}{3} = .67 \\ P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5 & P(\text{Sam} | \text{am}) = \frac{1}{2} = .5 & P(\text{do} | \text{I}) = \frac{1}{3} = .33 \end{array}$$

Neural Language Models



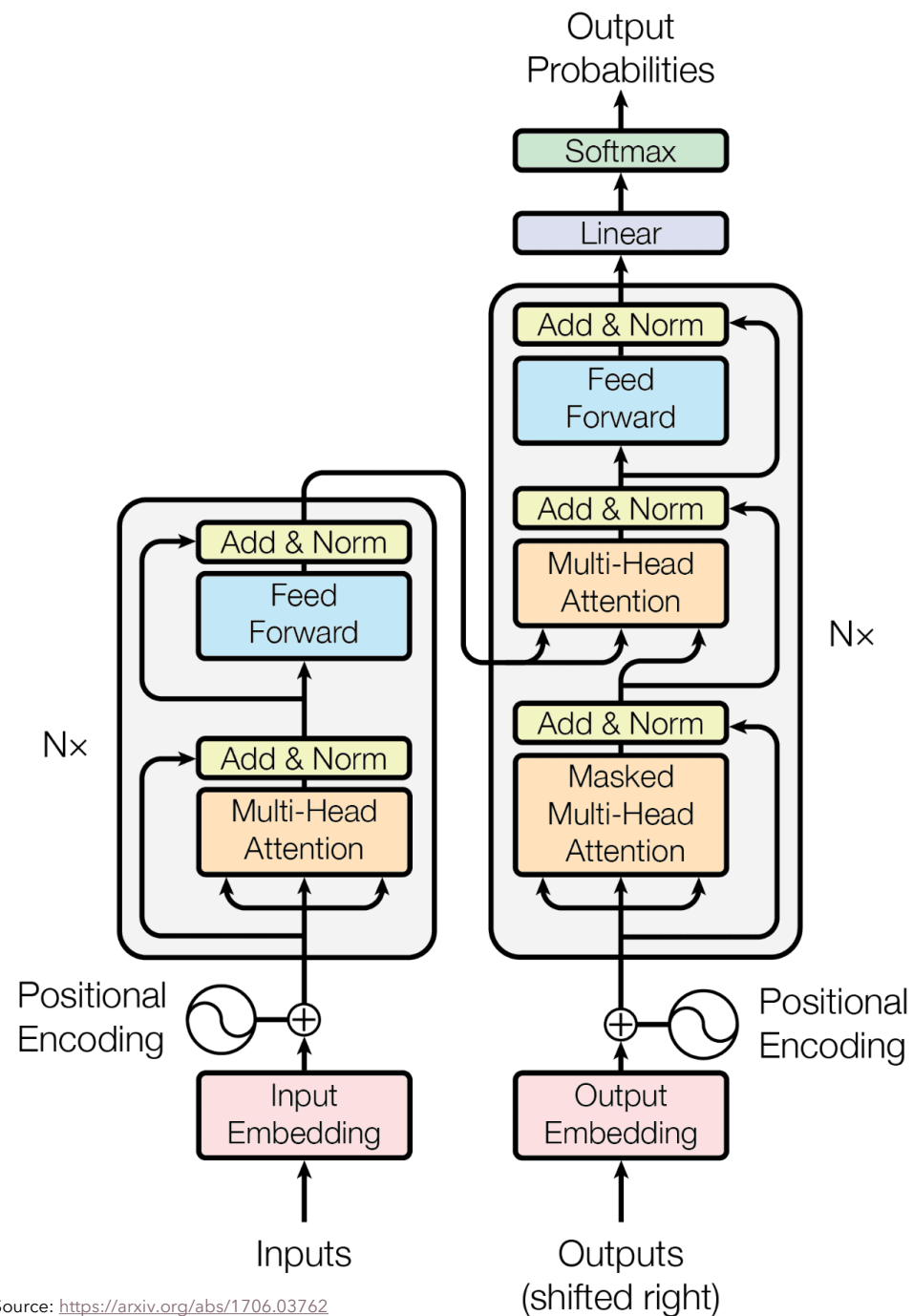
Recurrent Neural Networks

- Linear processing
- Recurrent Connections
 - Self-Looping Mechanism
 - Memory Retention
- Variable-Length Sequences
 - Flexible Processing
- Limitations
 - Vanishing Gradient Problem
 - Difficulty with Long-Range Dependencies
- Important Extension: Attention layer



Transformer

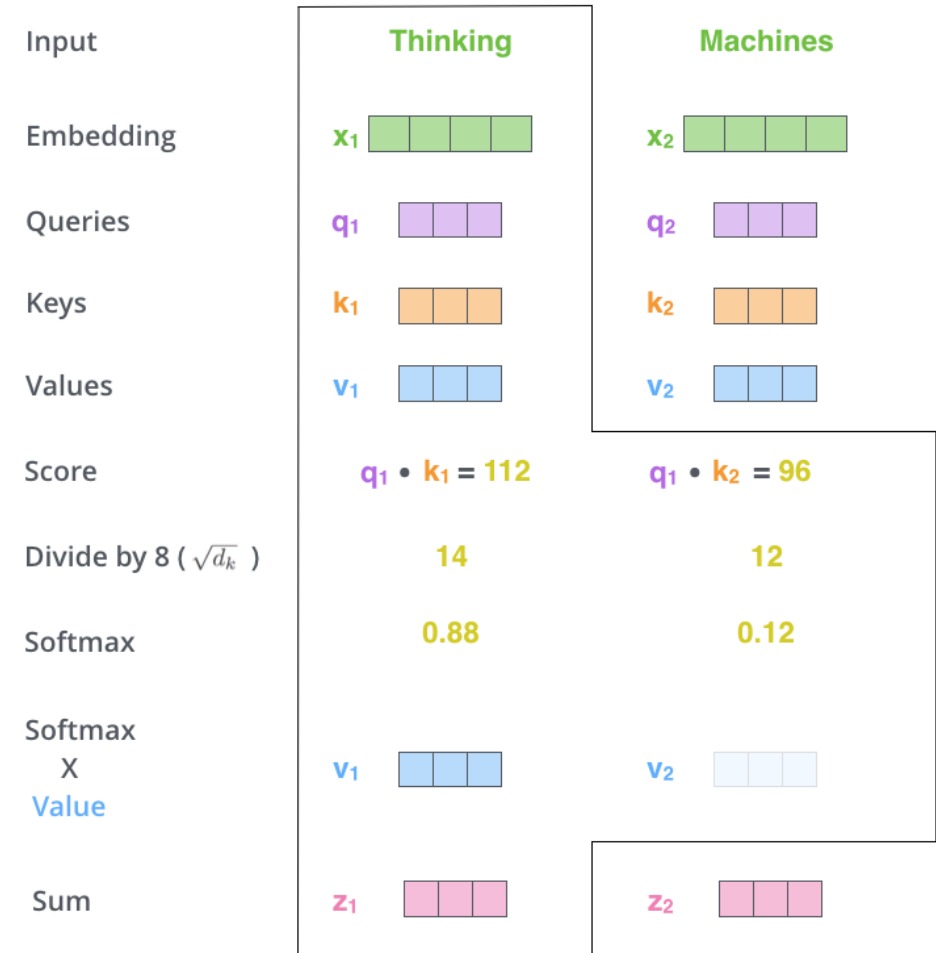
- Attention is all you need (2017)
- How information flows through?
- State-of-the-art attention-based model
 - Many variants: BERT (encoder, for classification), GPT (decoder, for generation), T5 (generation)
- **Multi-head Attention:** Contextual embeddings
- Limitations
 - Fixed size input: 256 to 128k tokens
- Can get very large
 - Large LLMs have approx. 175B parameters



Attention Mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Attention mechanism allows the model to **focus on specific parts of the input sequence**.
- Model dynamically weights the importance of different parts of the input.
- How it works?
 - **Input Representation:** Each word in the input sequence is represented as a vector (embedding).
 - **Scoring:** The attention mechanism computes a score that represents the relevance of each input word to the current output word being generated.
 - **Context Vector:** A weighted sum of input vectors is calculated using these weights, resulting in a context vector that emphasizes relevant parts of the input.

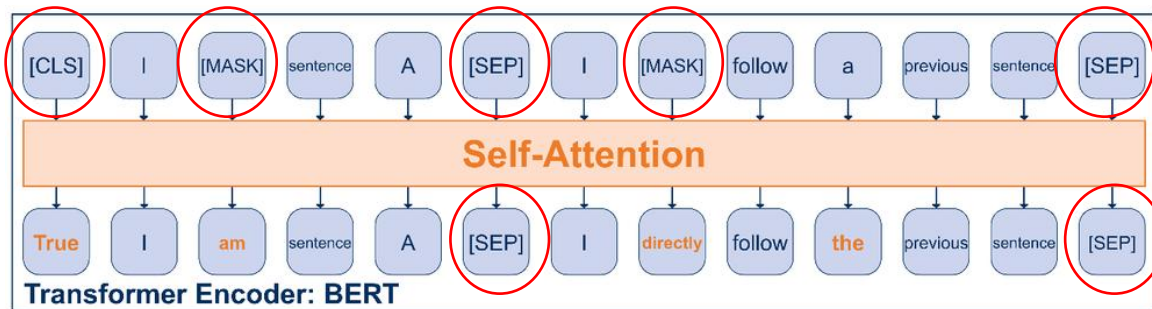


Contextual Embedding



Tokenization (building Vocabulary)

- Create tokens (e.g. sentencepiece, byte-pair encoding)
- 100 tokens $\approx$ 75 words
- <https://platform.openai.com/tokenizer>
- Vocabularies are usually very large, approx. 50k-200k in size
- Special tokens
 - **[CLS]**: Used for classification tasks in some models.
 - **[SEP]**: Used to separate segments or sentences in some models.
 - **[MASK]**: Used for masking tokens during pre-training or for applying a masked language model objective.



Pipeline language models

1. Step 1: Build tokenizer

2. Step 2: Train language model (pre-training)

- Trained on a vast amount of data with **self-supervised approach**: Model predicts the missing or next word
- Model "get's" **the basic idea of what is language** through probability distribution over its vocabulary

3. Optional Step 3: adapt for specific task (fine-tuning)

- Sentiment analysis, summarization, machine-translation, chat
- We can freeze parameters and train only task-specific head on top of the model